

Backend Schema Document — Sustainable Clothing Platform

1. Workflow Recap (so schema decisions make sense)

Tier	Input	Action	Output
Tier 1 - Resale	Wearable clothes that don't fit anymore	Listed on site, another user buys	Platform takes $\geq 10\%$ commission on sale
Tier 2 - Recycle	Completely worn-out clothes	Collected, sent to a partner factory	Factory converts fabric $\rightarrow$ thread/raw material
Tier 3 - Upcycle	Worn-out clothes/bedsheets etc.	Given to a tailor partner, customized into a new product	New product (e.g. pillow from bedsheet) listed for sale

Every item enters through **one funnel** (a single "Item" submission), then a classification step routes it into one of the three tier-specific pipelines. This is the key design decision below: one core `Item`, three tier-specific child tables, so you don't duplicate user/photo/address logic three times.

2. Entity List (Collections)

2.1 `users`

Core identity for everyone — sellers, buyers, tailors, and factory partners can all originate from this table, differentiated by `role`.

```
js
{
  _id: ObjectId,
  name: String,
  email: String,           // unique, indexed
  phone: String,          // unique, indexed
  passwordHash: String,
  role: String,            // enum: "customer" | "tailor_partner" | "factory_partner"
  rating: Number,          // avg rating, default 0
  ratingCount: Number,
  walletBalance: Number,   // for seller payouts pending withdrawal
  createdAt: Date,
  updatedAt: Date
}
```

2.2 `addresses`

Separated from `users` because pickups happen from different addresses across items.

```
js
{
  _id: ObjectId,
  userId: ObjectId,      // ref: users
  line1: String,
  line2: String,
  city: String,
  state: String,
  pincode: String,
  isDefault: Boolean
}
```

2.3 `items` (the core submission — every tier starts here)

```
js
{
  _id: ObjectId,
  ownerId: ObjectId,      // ref: users (the person submitting)
  category: String,       // "shirt" | "jeans" | "bedsheet" | "saree" | etc.
  title: String,
  description: String,
  images: [String],       // uploaded photo URLs
  condition: String,      // "wearable" | "worn_out"
  tier: Number,            // 1, 2, or 3 – set after admin/AI classification
  status: String,         // see status flow below (per-tier), shared field
  pickupRequestId: ObjectId, // ref: pickup_requests, nullable until scheduled
  createdAt: Date,
  updatedAt: Date
}
```

`tier` decides which of the next three collections gets a matching document. Keep `condition` separate from `tier` — condition is what the user reports, tier is what your team/algorithm decides (a "worn_out" bedsheet could go to Tier 2 or Tier 3 depending on tailor feasibility).

2.4 Tier 1 — `resale_listings`

```
js
```

```

{
  _id: ObjectId,
  itemId: ObjectId,           // ref: items, 1:1
  sellerId: ObjectId,         // ref: users (redundant but useful for fast queri
  price: Number,
  commissionPercent: Number, // default 10, admin can override per listing
  status: String,             // "listed" | "sold" | "delisted"
  buyerId: ObjectId,         // ref: users, set on sale
  soldAt: Date,
  createdAt: Date
}

```

2.5 Tier 2 — `recycling_batches` + `factories`

Items don't get recycled one at a time — they're batched and shipped to a factory. So this is a **many-to-many**: many items → one batch → one factory.

```

js

// factories (partner directory)
{
  _id: ObjectId,
  name: String,
  contactPerson: String,
  phone: String,
  location: String,
  materialTypesAccepted: [String], // "cotton" | "polyester" | "mixed"
  createdAt: Date
}

// recycling_batches
{
  _id: ObjectId,
  factoryId: ObjectId,           // ref: factories
  itemIds: [ObjectId],          // ref: items (many)
  totalWeightKg: Number,
  status: String,               // "collected" | "shipped" | "processed" | "closed
  shippedAt: Date,
  processedAt: Date,
  createdAt: Date
}

```

2.6 Tier 3 — `upcycle_requests`, `tailors`, `upcycled_products`

This tier has an extra step compared to the others: raw item → tailor work → **new sellable product**. So it needs its own mini resale table (`upcycled_products`), separate from `resale_listings`, because the seller here is the *platform*, not the original owner.

```
js

// tailors (partner directory)
{
  _id: ObjectId,
  name: String,
  phone: String,
  location: String,
  specialization: [String], // "pillow_making" | "bag_stitching" | "patchwork"
  createdAt: Date
}

// upcycle_requests
{
  _id: ObjectId,
  itemId: ObjectId,          // ref: items
  tailorId: ObjectId,        // ref: tailors
  requestedProductType: String, // "pillow" | "tote_bag" | "quilt" etc.
  beforeImages: [String],
  afterImages: [String],
  status: String,             // "assigned" | "in_progress" | "completed" | "re
  completedAt: Date,
  createdAt: Date
}

// upcycled_products (final sellable item – separate catalog from resale_listin
{
  _id: ObjectId,
  upcycleRequestId: ObjectId, // ref: upcycle_requests, 1:1
  productType: String,
  price: Number,
  images: [String],
  status: String,             // "listed" | "sold" | "delisted"
  buyerId: ObjectId,
  soldAt: Date,
  createdAt: Date
}
```

2.7 Shared: `orders`, `payments`

Both Tier 1 and Tier 3 result in a sale, so orders/payments are shared — differentiated by `productType`.

```
js
// orders
{
  _id: ObjectId,
  buyerId: ObjectId,      // ref: users
  productType: String,    // "resale_listing" | "upcycled_product"
  productRefId: ObjectId, // ref: resale_listings OR upcycled_products
  amount: Number,
  commissionAmount: Number, // platform's cut
  sellerPayout: Number,    // amount - commission (0 for upcycled since platfo
  status: String,          // "pending" | "paid" | "shipped" | "delivered" | "
  createdAt: Date
}

// payments
{
  _id: ObjectId,
  orderId: ObjectId,      // ref: orders, 1:1
  method: String,         // "upi" | "card" | "netbanking"
  transactionId: String,
  status: String,         // "success" | "failed" | "refunded"
  paidAt: Date
}
```

2.8 `pickup_requests`

Covers logistics for all three tiers — items get physically collected before anything else happens.

```
js
```

```

{
  _id: ObjectId,
  userId: ObjectId,
  itemIds: [ObjectId],      // items being picked up together
  addressId: ObjectId,
  scheduledDate: Date,
  status: String,           // "requested" | "scheduled" | "picked_up" | "canceled"
  createdAt: Date
}

```

2.9 **reviews** (optional but recommended)

```

js
{
  _id: ObjectId,
  reviewerId: ObjectId,
  targetType: String,      // "seller" | "tailor" | "factory"
  targetId: ObjectId,
  rating: Number,          // 1-5
  comment: String,
  createdAt: Date
}

```

3. Relationship Map

```

users (1) —< items (M)                [one user submits many items]
users (1) —< addresses (M)
items (1) — (1) resale_listings        [Tier 1]
items (M) >—< recycling_batches (1)    [Tier 2, via itemIds array]
factories (1) —< recycling_batches (M)
items (1) — (1) upcycle_requests (1) — (1) upcycled_products [Tier 3 chain]
tailors (1) —< upcycle_requests (M)
users (1) —< orders (M) [as buyer]
orders (1) — (1) payments (1)
users (1) —< pickup_requests (M)

```

4. Status Flow Per Tier (state machine — important for your admin dashboard)

- **Tier 1:** `submitted → under_review → listed → sold / delisted`

- **Tier 2:** `submitted` → `under_review` → `collected` → `shipped` → `processed`
- **Tier 3:** `submitted` → `under_review` → `assigned_to_tailor` → `in_progress` → `completed` → `listed (as upcycled_product)` → `sold`

The `items.status` field should mirror the current stage; keep the tier-specific tables as the source of truth and update `items.status` via a hook/middleware whenever the child table changes — avoids the two getting out of sync.

5. Indexing Notes (for when you build this in Mongoose)

- `users.email`, `users.phone` → unique index
- `items.ownerId`, `items.tier`, `items.status` → compound index (admin dashboard filters by these constantly)
- `resale_listings.status`, `upcycled_products.status` → index (public browse/search pages)
- `orders.buyerId`, `orders.status` → compound index

6. Notes on the 10% Commission Logic

Keep `commissionPercent` on the listing itself (not hardcoded in code) — lets you run promotions later (e.g. 5% for first-time sellers) without a schema change. Compute `sellerPayout` and `commissionAmount` at order-creation time and store both, rather than recalculating later — protects you if you change the commission rate in future and someone looks up an old order.